

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

LAB TASK 5

BECE406E

FPGA System Design Lab

NAME	KAARMUKILAN A G
REGISTRATION NO	23BEC0137
SLOT	L33+L34
FACULTY NAME	AARTHY M

AIM:

i) Design a 4 tap - serial FIR filter and 4 tap - pipelined FIR filter with filter coefficients of h0=4, h1=3, h2=2, h1=1. Verify the functionality using modelsim and justify which design is preferred. Implement only 4 tap - serial FIR filter in FPGA board. Attach screenshots of code and output, RTL view and FPGA board implementation.

ii) Design edge detection algorithm using Sobel operator and verify the functionality using modelsim with test cases showing no edge, strong edge (Horizontal/vertical), moderate edge. Attach Screenshots of code and outputs.

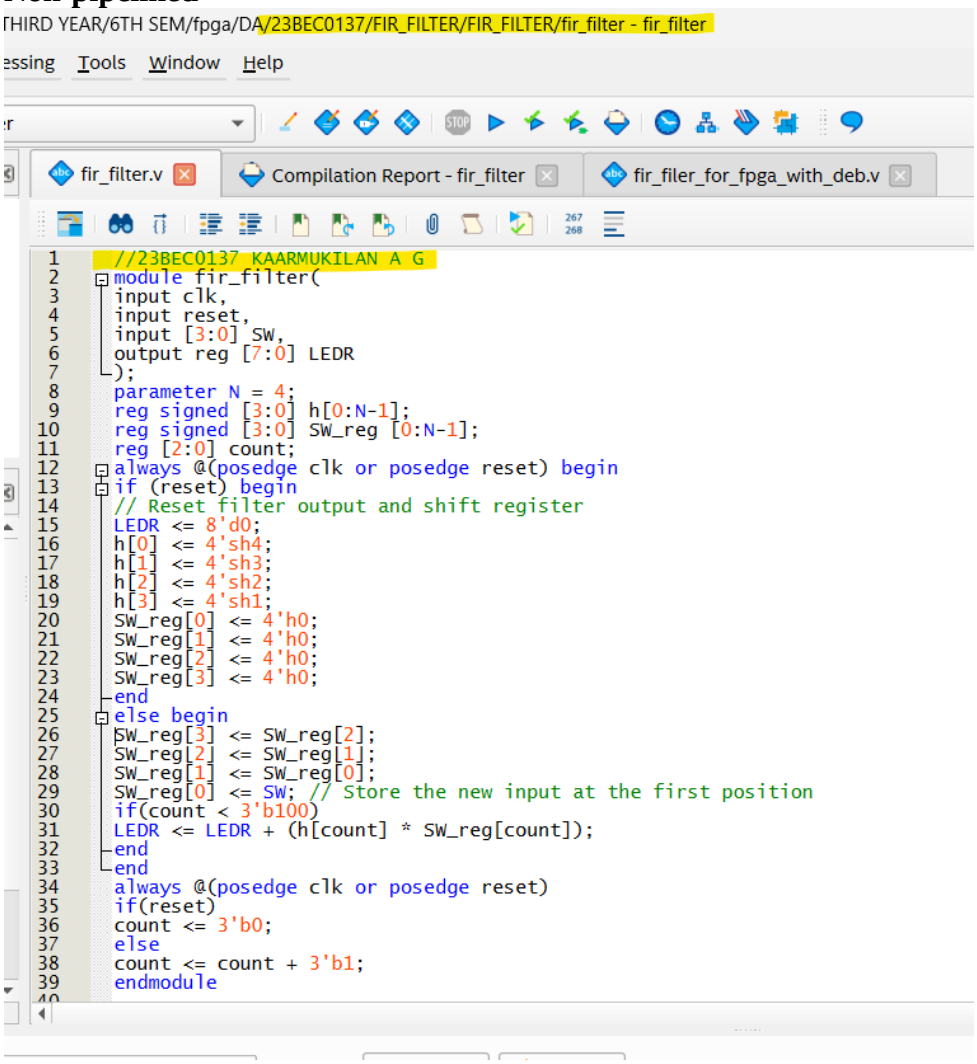
SOFTWARE DETAILS:

Functional Simulation: Modelsim
Synthesis & Implementation: Intel Quartus Prime 23.1

FIR FILTER

RTL CODE:

Non-pipelined




```
1  //23BEC0137 KAARMUKILAN A G
2  module fir_filter(
3  input clk,
4  input reset,
5  input [3:0] SW,
6  output reg [7:0] LEDR
7  );
8  parameter N = 4;
9  reg signed [3:0] h[0:N-1];
10 reg signed [3:0] SW_reg [0:N-1];
11 reg [2:0] count;
12 always @(posedge clk or posedge reset) begin
13 if (reset) begin
14 // Reset filter output and shift register
15 LEDR <= 8'd0;
16 h[0] <= 4'sh4;
17 h[1] <= 4'sh3;
18 h[2] <= 4'sh2;
19 h[3] <= 4'sh1;
20 SW_reg[0] <= 4'h0;
21 SW_reg[1] <= 4'h0;
22 SW_reg[2] <= 4'h0;
23 SW_reg[3] <= 4'h0;
24 end
25 else begin
26 SW_reg[3] <= SW_reg[2];
27 SW_reg[2] <= SW_reg[1];
28 SW_reg[1] <= SW_reg[0];
29 SW_reg[0] <= SW; // Store the new input at the first position
30 if(count < 3'b100)
31 LEDR <= LEDR + (h[count] * SW_reg[count]);
32 end
33 end
34 always @(posedge clk or posedge reset)
35 if(reset)
36 count <= 3'b0;
37 else
38 count <= count + 3'b1;
39 endmodule
```

Pipelined

E:/r4R9lC#e4/THIRD YEAR/6TH SEM/fpga/DA/23BEC0137/FIR_FILTER/FIR_FILTER/fir_filter.v (

```
.n#
 1    //23BEC0137 KAARMUKILAN A G
 2   module fir_filter(
 3    input clk,
 4    input reset,
 5    input [3:0] SW,
 6    output reg [7:0] LEDR
 7    );
 8    parameter N = 4;
 9    reg signed [3:0] h[0:N-1];
10    reg signed [3:0] SW_reg [0:N-1];
11    //  FIXED WIDTH
12    reg signed [7:0] mult [0:N-1];
13    reg signed [7:0] sum1, sum2;
14   always @(posedge clk or posedge reset) begin
15   if (reset) begin
16        LEDR <= 0;
17        h[0] <= 4'sd4;      h[1] <= 4'sd3;
18        h[2] <= 4'sd2;      h[3] <= 4'sd1;
19        SW_reg[0] <= 0;      SW_reg[1] <= 0;
20        SW_reg[2] <= 0;      SW_reg[3] <= 0;
21        mult[0] <= 0;    mult[1] <= 0;
22        mult[2] <= 0;    mult[3] <= 0;
23        sum1 <= 0;    sum2 <= 0;
24    end
25   else begin
26        // Stage 1: shift register
27        SW_reg[3] <= SW_reg[2];
28        SW_reg[2] <= SW_reg[1];
29        SW_reg[1] <= SW_reg[0];
30        SW_reg[0] <= SW;
31        // Stage 2: multiplication
32        mult[0] <= h[0] * SW_reg[0];
33        mult[1] <= h[1] * SW_reg[1];
34        mult[2] <= h[2] * SW_reg[2];
35        mult[3] <= h[3] * SW_reg[3];
36        // Stage 3: partial sums
37        sum1 <= mult[0] + mult[1];
38        sum2 <= mult[2] + mult[3];
39        // Stage 4: final sum
40        LEDR <= sum1 + sum2;
41    end
42    end
43    endmodule
44
```

Pipelined is Preferred.

TESTBENCH:

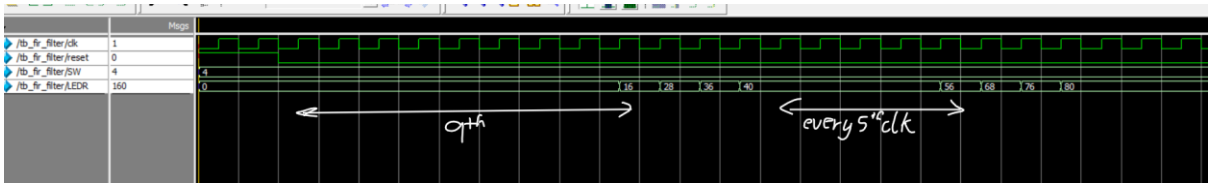
lC#e4/THIRD YEAR/6TH SEM/fpga/DA/23BEC0137/FIR_FILTER/fir_filter_tb_23BEC0137.v - Default

```
 `timescale 1ns/1ps
module tb_fir_filter;
reg clk;reg reset;
reg [3:0] SW;
wire [7:0] LEDR;

// DUT
fir_filter uut (
    .clk(clk),
    .reset(reset),
    .SW(SW),
    .LEDR(LEDR)
);
// Clock
always #5 clk = ~clk;
initial begin
    clk = 0;
    reset = 1;
    SW = 4'd4;
    $monitor("Time=%0t | SW=%d | LEDR=%d", $time, SW, LEDR);
    #20 reset = 0
    #200;
    $display("Done");
    $finish;
end
endmodule
```

FUNCTIONAL SIMULATION:

Non-pipelined



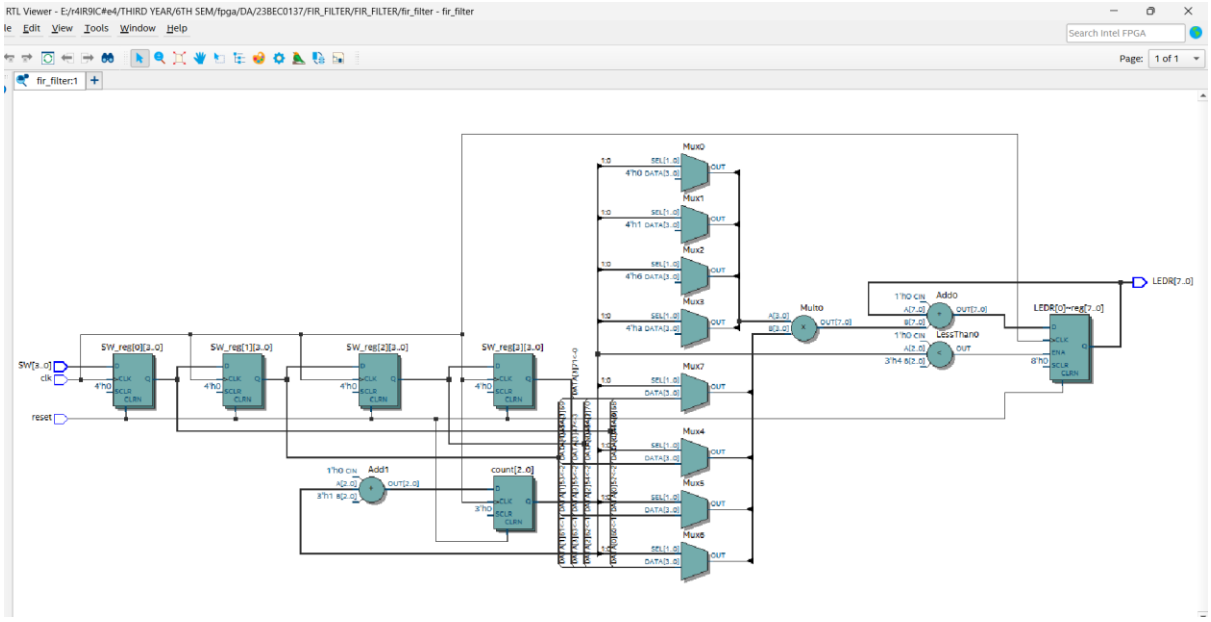
Pipelined



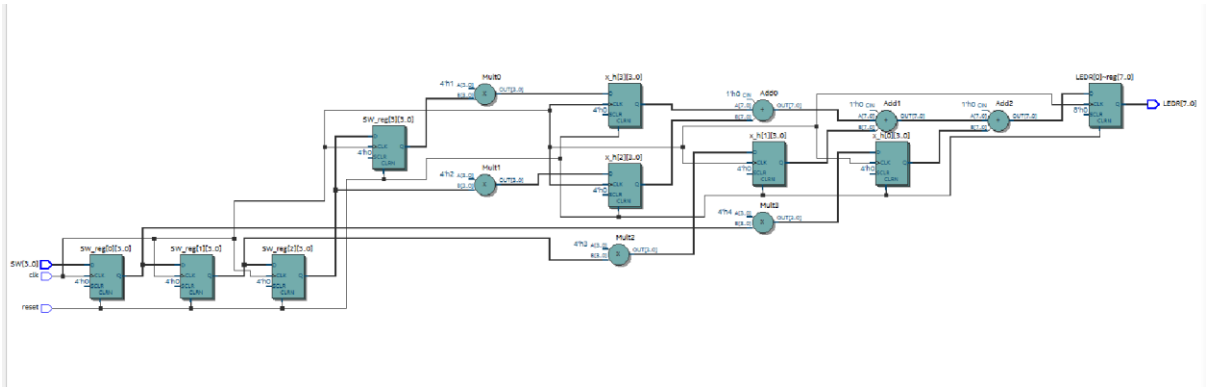
Pipelined takes lesser clock cycles than serial case

RTL VIEW:

Non-pipelined



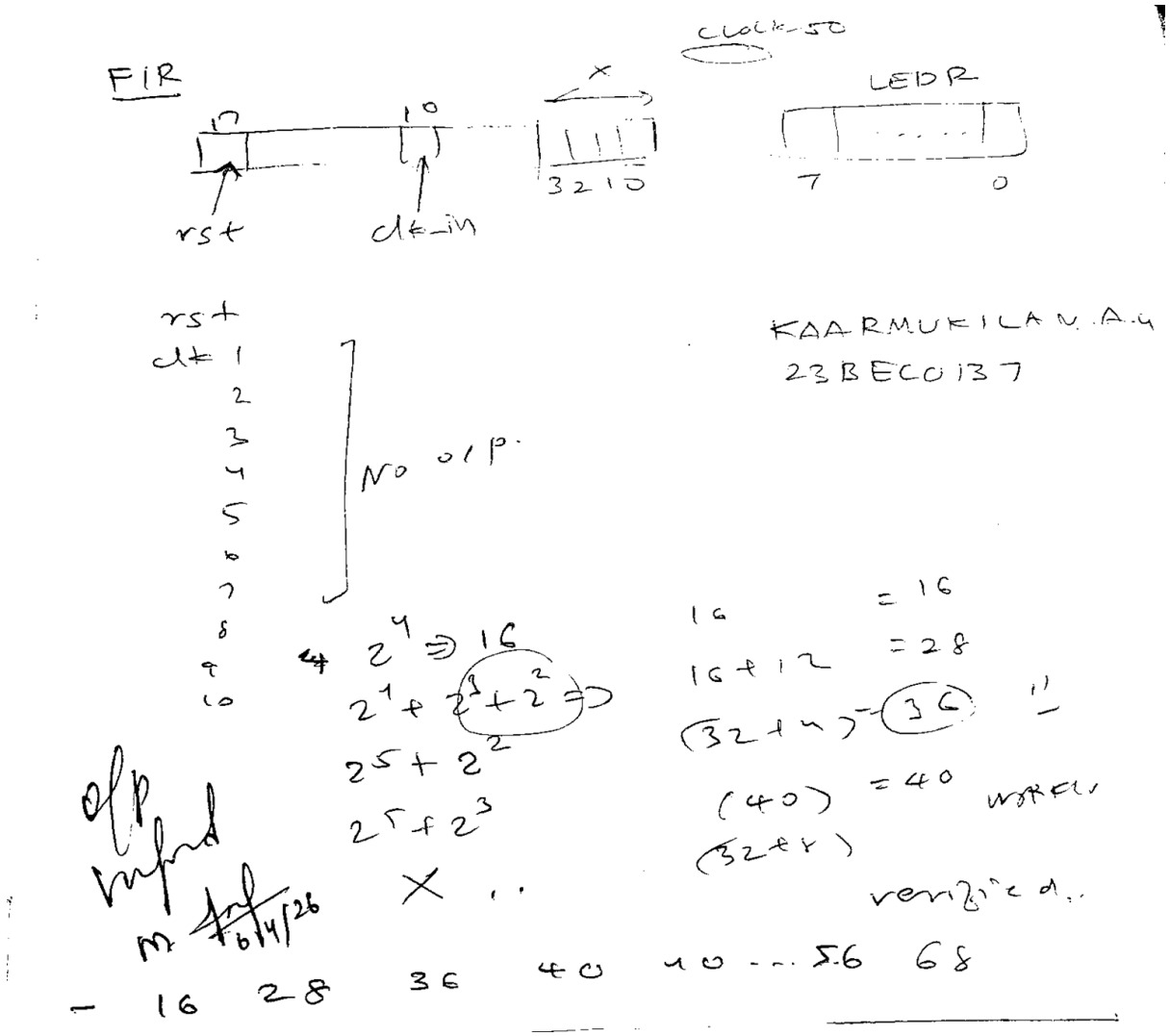
Pipelined



FPGA IMPLEMENTATION (4-TAP SERIAL)



ATTESTATION:



SOBEL EDGE FILTER 3X3

RTL CODE :

E:/r4lR9lC#e4/THIRD YEAR/6TH SEM/fpga/DA/23BEC0137/SOBEL_eDGE_3/sobel_filter.v (/tb_sobel_edge_detection) - Default

```
Ln#
  1   //23BEC0137 KAARMUKILAN A G
  2   module sobel_edge_detection #(parameter WIDTH = 8) (
  3       input wire clk,
  4       input wire rst,
  5       input wire [WIDTH-1:0] pixel_in,
  6       input wire [WIDTH-1:0] pixel_in_top,
  7       input wire [WIDTH-1:0] pixel_in_bottom,
  8       input wire [WIDTH-1:0] pixel_in_left,
  9       input wire [WIDTH-1:0] pixel_in_right,
 10       input wire [WIDTH-1:0] pixel_in_tl,
 11       input wire [WIDTH-1:0] pixel_in_tr,
 12       input wire [WIDTH-1:0] pixel_in_bl,
 13       input wire [WIDTH-1:0] pixel_in_br,
 14       output reg [WIDTH-1:0] edge_out,
 15       output reg ready
 16   );
 17
 18   // Sobel kernels
 19   reg signed [WIDTH-1:0] sobel_x [0:2][0:2];
 20   reg signed [WIDTH-1:0] sobel_y [0:2][0:2];
 21
 22   // Gradients
 23   reg signed [15:0] gradient_x;
 24   reg signed [15:0] gradient_y;
 25   reg signed [15:0] gradient_magnitude;
 26
 27   //initialise kernels
 28   initial begin
 29       sobel_x[0][0] = -1; sobel_x[0][1] = 0; sobel_x[0][2] = 1;
 30       sobel_x[1][0] = -2; sobel_x[1][1] = 0; sobel_x[1][2] = 2;
 31       sobel_x[2][0] = -1; sobel_x[2][1] = 0; sobel_x[2][2] = 1;
 32
 33       sobel_y[0][0] = -1; sobel_y[0][1] = -2; sobel_y[0][2] = -1;
 34       sobel_y[1][0] = 0;  sobel_y[1][1] = 0;  sobel_y[1][2] = 0;
 35       sobel_y[2][0] = 1;  sobel_y[2][1] = 2;  sobel_y[2][2] = 1;
 36   end
 37
 38   always @(posedge clk or posedge rst) begin
 39       if (rst) begin
```

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        gradient_x <= 0;
        gradient_y <= 0;
        gradient_magnitude <= 0;
        edge_out <= 0;
        ready <= 0;
    end
    else begin
        // X gradient
        gradient_x <= (sobel_x[0][0] * pixel_in_tl) + (sobel_x[0][1] * pixel_in_top) + (sobel_x[0][2] * pixel_in_tr) +
                      (sobel_x[1][0] * pixel_in_left) + (sobel_x[1][1] * pixel_in) + (sobel_x[1][2] * pixel_in_right) +
                      (sobel_x[2][0] * pixel_in_bl) + (sobel_x[2][1] * pixel_in_bottom) + (sobel_x[2][2] * pixel_in_br);

        // Y gradient
        gradient_y <= (sobel_y[0][0] * pixel_in_tl) + (sobel_y[0][1] * pixel_in_top) + (sobel_y[0][2] * pixel_in_tr) +
                      (sobel_y[1][0] * pixel_in_left) + (sobel_y[1][1] * pixel_in) + (sobel_y[1][2] * pixel_in_right) +
                      (sobel_y[2][0] * pixel_in_bl) + (sobel_y[2][1] * pixel_in_bottom) + (sobel_y[2][2] * pixel_in_br);

        // Magnitude (approx)
        gradient_magnitude <= (gradient_x * gradient_x) + (gradient_y * gradient_y);

        // Output scaling
        edge_out <= (gradient_magnitude[15:8] > 8'd255) ? 8'd255 : gradient_magnitude[15:8];

        ready <= 1;
    end
end

endmodule
```

TESTBENCH:

```
`timescale 1ns / 1ps

module tb_sobel_edge_detection;

parameter WIDTH = 8;

reg clk;
reg rst;
reg [WIDTH-1:0] pixel_in, pixel_in_top, pixel_in_bottom;
reg [WIDTH-1:0] pixel_in_left, pixel_in_right;
reg [WIDTH-1:0] pixel_in_tl, pixel_in_tr, pixel_in_bl, pixel_in_br;

wire [WIDTH-1:0] edge_out;
wire ready;

sobel_edge_detection #(WIDTH) uut (
    .clk(clk), .rst(rst),
    .pixel_in(pixel_in),
    .pixel_in_top(pixel_in_top),
    .pixel_in_bottom(pixel_in_bottom),
    .pixel_in_left(pixel_in_left),
    .pixel_in_right(pixel_in_right),
    .pixel_in_tl(pixel_in_tl),
    .pixel_in_tr(pixel_in_tr),
    .pixel_in_bl(pixel_in_bl),
    .pixel_in_br(pixel_in_br),
    .edge_out(edge_out),
    .ready(ready)
);

// Clock
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;

    //  Monitor output
    $monitor("Time=%0t | Edge_out=%d | Ready=%b", $time, edge_out, ready);

    #20 rst = 0;
    #10;

    // Test Case 1: No Edge

    // Test Case 1: No Edge
    $display("Test Case 1: No Edge");
    pixel_in = 100; pixel_in_top = 100; pixel_in_bottom = 100;
    pixel_in_left = 100; pixel_in_right = 100;
    pixel_in_tl = 100; pixel_in_tr = 100; pixel_in_bl = 100; pixel_in_br = 100;
    #50;

    // Test Case 2: Vertical Edge (Strong)
    $display("Test Case 2: Strong Vertical Edge");
    pixel_in_tl = 10; pixel_in_top = 10; pixel_in_tr = 200;
    pixel_in_left = 10; pixel_in = 10; pixel_in_right = 200;
    pixel_in_bl = 10; pixel_in_bottom = 10; pixel_in_br = 200;
    #50;

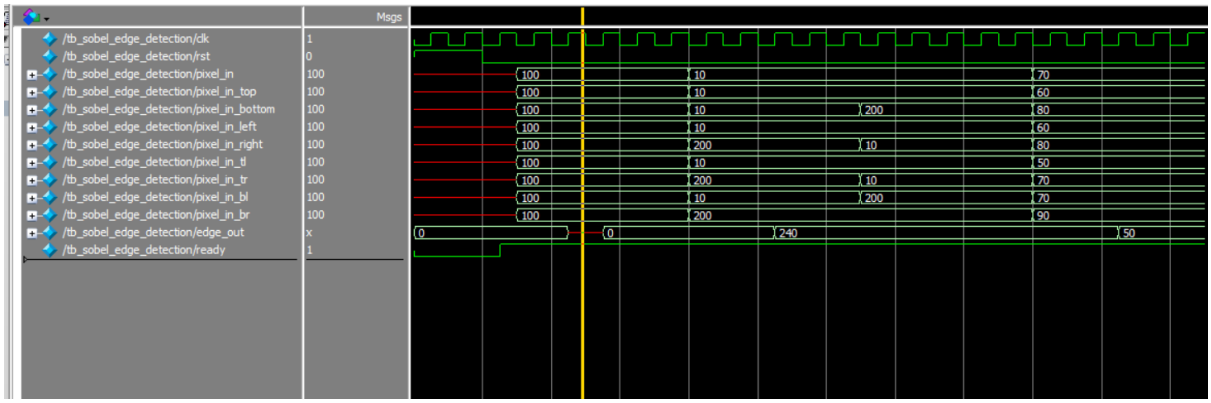
    // Test Case 3: Horizontal Edge (Strong)
    $display("Test Case 3: Strong Horizontal Edge");
    pixel_in_tl = 10; pixel_in_top = 10; pixel_in_tr = 10;
    pixel_in_left = 10; pixel_in = 10; pixel_in_right = 10;
    pixel_in_bl = 200; pixel_in_bottom = 200; pixel_in_br = 200;
    #50;

    // Test Case 4: Moderate Edge
    $display("Test Case 4: Moderate Edge");
    pixel_in_tl = 50; pixel_in_top = 60; pixel_in_tr = 70;
    pixel_in_left = 60; pixel_in = 70; pixel_in_right = 80;
    pixel_in_bl = 70; pixel_in_bottom = 80; pixel_in_br = 90;
    #50;

    $display("Simulation Complete.");
    $finish;
end

endmodule
```

FUNCTIONAL SIMULATION:



Last case – Moderate edge

```
# Time=0 | Edge_out=  0 | Ready=0
# Time=25000 | Edge_out=  0 | Ready=1
# Test Case 1: No Edge
# Time=45000 | Edge_out=  x | Ready=1
# Time=55000 | Edge_out=  0 | Ready=1
# Test Case 2: Strong Vertical Edge
# Time=105000 | Edge_out=240 | Ready=1
# Test Case 3: Strong Horizontal Edge
# Test Case 4: Moderate Edge
# Time=205000 | Edge_out= 50 | Ready=1
# Simulation Complete.
# ** Note: $finish    : E:/r41R91C#e4/THIRD YEAR/6TH SEM/fpga/DA/23BEC0137/SOBEL_eDGE_3/sobel_filter.v(141)
#    Time: 230 ns  Iteration: 0  Instance: /tb_sobel_edge_detection
# 1
```

INFERENCE:

Hence, FIR filter implementation has been done, and Sobel Edge detection simulation has been done.

- Pipelined FIR filter is preferred over Non pipelined.